

O que você descreveu aponta menos para “aprender programação” no sentido tradicional e mais para descobrir **novos meios computacionais de criação**.

O Three.js foi revelador porque juntou coisas que antes estavam separadas no seu repertório: 3D, animação, design, lógica, interação e IA. A descoberta importante é que existem várias outras tecnologias com essa mesma propriedade.

A mudança de paradigma: o navegador virou um estúdio multimídia programável

Sua comparação com Flash faz bastante sentido, mas a infraestrutura atual é radicalmente mais ampla. Hoje o navegador reúne GPU via WebGL/WebGPU, áudio procedural via Web Audio, aplicações complexas em JavaScript/TypeScript, WebAssembly, APIs de câmera/microfone, networking em tempo real e engines 2D/3D. WebGPU, inclusive, não serve apenas para gráficos: expõe computação geral na GPU. [MDN Web Docs](#)

A IA adicionou a peça que faltava: **você não precisa dominar completamente a sintaxe de cada meio para começar a operá-lo**.

Isso transforma algo como:

ideia → software complexo → especialista → resultado

em algo mais próximo de:

ideia → especificação → IA escreve o sistema → você dirige visualmente → IA refatora → resultado

Seu background em After Effects, Cinema 4D e design deixa de ser apenas experiência passada. Ele fornece os modelos mentais necessários para dirigir o código: timeline, easing, câmera, hierarquia, pivot, world/local coordinates, materiais, rigging, composição, partículas, física etc.

Eu chamaria a região profissional para a qual você está convergindo de **Creative Technology / Computational Design / AI-native Creative Engineering**.

As pontes que eu investigaria

Ponte	Tecnologias	O que ela acrescenta ao seu repertório	Prioridade
3D web avançado	Three.js + React Three Fiber + Drei + Rapier	transforma 3D em componentes reutilizáveis e aplicações	★★★★★
Shaders/GPU	GLSL → WGSL → WebGPU	efeitos procedurais, partículas, materiais, compute	★★★★★
Motion programático	Remotion + Motion + SVG	After Effects dirigido por código/dados/IA	★★★★★
Creative coding 2D	PixiJS + p5.js	o lado mais “Flash” da web moderna	★★★★★
Música no browser	Tone.js + Strudel	beats, sequencers, audiovisual generativo	★★★★★
Síntese sonora profunda	SuperCollider + Faust	instrumentos, DSP, síntese e áudio procedural	★★★★☆
Design AI-native	Penpot + MCP + SVG/CSS	design editável simultaneamente por humano e agente	★★★★★
Games/apps interativos	Godot	quando Three.js deixa de ser suficiente como engine	★★★★☆
Agentes visuais	Langflow / Dify / n8n	prototipagem visual de sistemas agentic	★★★★☆
Agentes code-first	Pydantic AI / LangGraph	arquitetura séria para algo como Hermes Work	★★★★★
Browser agents	Playwright / Browser Use	execução real no navegador	★★★★★

Há alguns pontos especialmente importantes aqui.

1. Three.js é só o começo: React Three Fiber é provavelmente sua próxima descoberta

React Three Fiber não é uma engine concorrente: é um renderer React para Three.js. O mesmo `mesh`, câmera, material e geometria passa a existir dentro de uma árvore React, reagindo a estado e eventos. A documentação inclusive afirma que os recursos do Three.js continuam disponíveis. [Poimandres Documentação](#)

Isso significa que você sai de:

```
site
+
experiência 3D
```

para:

```
aplicação
├─ interface React
├─ dados
├─ estado
├─ IA
└─ mundo Three.js
```

Imagine, por exemplo, o dashboard da ACIRV não apresentando apenas gráficos.

Uma IA analisa os dados e constrói uma **visualização espacial navegável**, em que publicações são objetos, tamanho representa alcance, proximidade representa similaridade semântica, cores representam clusters editoriais e o usuário literalmente atravessa os dados.

Isso já é outra categoria de mídia.

2. O verdadeiro próximo salto depois de Three.js é shader programming

Eu colocaria **GLSL/WGSL** muito acima de aprender uma nova linguagem genérica aleatória.

Quando você programa shaders, passa a programar diretamente a aparência da matéria digital:

```
posição
cor
luz
textura
ruído
distorção
partículas
refração
dissolve
fogo
água
glitch
```

campos
proceduralismo

E isso combina extraordinariamente com IA porque shaders são pequenos programas altamente parametrizáveis.

Seu caminho pode ser:

```
Three.js  
  ↓  
ShaderMaterial  
  ↓  
GLSL  
  ↓  
Node Materials / TSL  
  ↓  
WebGPU / WGSL
```

O Three.js já possui infraestrutura WebGPU, enquanto o ecossistema React Three Fiber está trabalhando sua camada WebGPU em sua próxima geração. [Three.js](#)

Aqui existe uma ponte muito clara com o que você conhecia de Cinema 4D: muitos “efeitos” que aparecem como plugins, deformers ou materiais nodais em software 3D são essencialmente **programas matemáticos encapsulados por uma interface**.

Agora você pode escrever esses programas.

3. Para você, Remotion pode ser tão importante quanto Three.js

Three.js faz:

código → espaço

Remotion faz:

código → tempo

O conceito fundamental do Remotion é literalmente tratar vídeo como uma função dos frames usando React. [Remotion](#)

E ele aceita dados como parâmetros do vídeo. [Remotion](#)

Isso permite algo bastante diferente de After Effects:

```
planilha
  ↓
dados
  ↓
React
  ↓
Remotion
  ↓
100 vídeos personalizados
```

Ou:

```
Relatório Instagram
  ↓
IA identifica insights
  ↓
gera roteiro
  ↓
gera gráficos
  ↓
gera composição Remotion
  ↓
renderiza Reel
```

Mais interessante ainda: em 2026 o próprio Remotion já oferece integração e skills voltadas especificamente a coding agents. [Remotion](#)

Isso encaixa quase perfeitamente no Hermes Work.

Você poderia chegar a algo como:

```
Hermes Work
  ↓
"Fça um Reel sobre os resultados de setembro"
  ↓
busca dados
  ↓
seleciona narrativa
  ↓
abre projeto Remotion
```

```
↓
gera composição
↓
usa Three.js numa cena
↓
gera áudio
↓
renderiza
↓
entrega MP4
```

A fronteira entre **software de automação** e **estúdio criativo** começa a desaparecer.

4. Para ressuscitar o espírito do Flash: conheça PixiJS

Three.js não é necessariamente a melhor resposta para tudo.

Para experiências essencialmente 2D, PixiJS é provavelmente uma descoberta muito relevante para você.

Hoje ele se apresenta justamente como engine para **games, apps e conteúdo digital interativo**, usando renderização acelerada por WebGL/WebGPU. [PixiJS](#)

Então eu pensaria assim:

```
DOM/CSS
interfaces tradicionais

↓

Motion / Anime.js
interfaces altamente animadas

↓

PixiJS
experiências 2D, jogos, partículas, canvas

↓

Three.js
3D
```



WebGPU
controle de baixo nível / compute

Esse continuum é muito mais útil do que escolher “uma ferramenta”.

5. p5.js: menos industrial, mas excelente para pensar visualmente em código

p5.js talvez pareça simples depois de Three.js, mas eu não o descartaria.

Ele foi construído especificamente em torno de **creative coding para artistas e designers**, incluindo experiências audiovisuais e interativas no navegador. [p5.js](#)

É excelente para pedir a uma IA coisas como:

Crie 20 experimentos diferentes de sistemas de partículas que representem conexão entre empresas.

E iterar visualmente.

O resultado interessante pode então migrar para Three.js, PixiJS, Remotion ou uma aplicação real.

Pense no p5.js como um **caderno de rascunho computacional**.

A ponte que talvez mais combine com sua história: música programável

Aqui existe uma sequência muito interessante.

Tone.js

É provavelmente seu melhor ponto de entrada.

Tone.js coloca uma camada musical sobre Web Audio e oferece transport, scheduling, synths e effects no navegador. [Tone.js](#)

Ou seja:

```
kick  
bass  
synth  
filter  
delay  
sequencer
```

podem existir dentro da mesma aplicação em que existem:

```
React  
Three.js  
Remotion  
WebGL
```

Isso torna possível construir algo como:

```
beat  
↓  
FFT/análise  
↓  
Three.js  
↓  
geometria reage ao áudio  
↓  
Remotion  
↓  
videoclipe
```

Tudo programaticamente.

Strudel

Aqui acho que você pode ter outro momento “Three.js”.

Strudel é uma plataforma open source para **live coding musical diretamente no navegador**, inspirada no TidalCycles. [Strudel](#)

Você escreve padrões como código e escuta imediatamente.

É extremamente compatível com IA porque composição musical passa a ser **texto estruturado**.

SuperCollider

Se Strudel é uma porta de entrada, SuperCollider é um universo.

Ele é simultaneamente engine de síntese e linguagem de composição algorítmica, com arquitetura client/server e possibilidade de ser controlado inclusive por Python, JavaScript e outras linguagens. [SuperCollider](#)

É onde eu iria se você eventualmente quiser criar:

```
sintetizadores próprios  
efeitos  
generative music  
sound design procedural  
instalações  
sistemas reativos
```

Faust

Faust vai ainda mais fundo.

É uma linguagem funcional especificamente voltada para DSP e síntese de áudio e pode compilar o mesmo processamento para C++, WebAssembly, Rust, aplicações web, plugins etc.

[Faust](#)

Então existe um caminho:

```
Strudel  
  ↓  
Tone.js / Web Audio  
  ↓  
SuperCollider  
  ↓  
Faust
```

Não tentaria aprender tudo imediatamente.

Penpot merece sua atenção por um motivo diferente

Você já conhece editores gráficos.

O interessante no Penpot agora não é simplesmente ser “uma alternativa open source ao Figma”.

É que o projeto está explicitamente desenvolvendo workflows para **designers + developers + AI agents**, inclusive com servidor MCP. [Penpot](#)

Isso muda sua relevância para o Hermes Work.

Imagine:

```
Hermes
↓
Penpot MCP
↓
cria layout
↓
você corrige visualmente
↓
Hermes lê as alterações
↓
gera React
↓
Motion anima
↓
Three.js adiciona profundidade
```

Esse é um fluxo que Photoshop ou Illustrator não foram arquitetados para oferecer.

E há uma camada intermediária que você não deveria ignorar: Motion

Para interfaces e motion design web sem precisar entrar em Canvas/Three.js, eu estudaria **Motion**.

Ele hoje cobre animações declarativas em React, gestures, layout animation, scroll, SVG etc., e é MIT/open source. Curiosamente, também está construindo tooling explicitamente voltado a agentes de programação. [Motion](#)

Para alguém vindo do After Effects:

```
After Effects expression
≈
função baseada em estado/tempo

keyframes
```

≈
keyframes/interpolation

precomp
≈
component

essential graphics
≈
props

expression controls
≈
state / parameters

A transição mental é menor do que parece.

Dify, Langflow, Flowise e n8n: eu faria uma distinção importante

Essas ferramentas não estão exatamente na mesma camada que Three.js.

Elas são **ambientes de composição de sistemas**, e não de mídia.

Dify continua oferecendo um ambiente visual para workflows, agentes, RAG, código, branching e human-in-the-loop, com Community Edition self-hostable. A própria Dify atualmente descreve essa edição como “source-available” com licença derivada de Apache 2.0, portanto eu evitaria tratá-la simplesmente como software Apache 2.0 puro. [Dify](#)

Langflow está particularmente interessante em 2026. A versão 1.11 adicionou human-in-the-loop, A2A e AG-UI; a linha 1.12 ganhou também versão Desktop. [Langflow](#)

E há uma informação importante para sua pesquisa: **eu não começaria hoje um projeto novo baseado em Flowise**. O projeto anunciou encerramento do desenvolvimento, arquivamento do repositório em agosto de 2026 e EOL em 31 de agosto de 2026. O código continua disponível sob Apache 2.0, mas o projeto oficial foi congelado. [Flowise](#)

Curiosamente, o próprio n8n também acabou de caminhar mais explicitamente para agentes: em 25 de setembro de 2026 anunciou uma nova arquitetura de Agents integrada aos workflows existentes. [n8n Blog](#)

Então hoje eu veria aproximadamente:

n8n
automação + integrações + agentes

Dify
produto AI + workflows + RAG + agentes

Langflow
laboratório visual de sistemas agentic

Pydantic AI / LangGraph
engenharia agentic code-first

Hermes Work
workstation/autonomia/computer use

Para o Hermes Work, eu iria progressivamente em direção a código, não a Dify

Essa talvez seja a conclusão mais importante em relação ao seu projeto.

Dify/Langflow podem ser excelentes **laboratórios conceituais**.

Mas a arquitetura de algo como Hermes Work tende a se beneficiar mais de uma camada code-first.

LangGraph é especificamente um runtime/orquestrador de agentes stateful de longa duração, com execução durável, persistência, streaming e human-in-the-loop, e permite misturar passos determinísticos com decisões de LLM. [LangChain AI](#)

E eu estudaria seriamente **Pydantic AI**.

A plataforma cresceu bastante: hoje inclui agents tipados, MCP, durable execution, subagents, planning, persistence, Browser Use, Playwright, evaluation e um harness para agentes de longa duração. [Pydantic Docs](#)

Para o Hermes, conceitos como:

typed tools
structured outputs
capabilities
execution environments
durable execution

```
evaluations
guardrails
browser tools
MCP
```

são muito mais valiosos do que simplesmente adicionar outro canvas de nodes.

E aqui existe outra ponte que você talvez esteja subestimando: o browser agent

Você está construindo algo que navega e executa trabalho.

Então eu trataria browser automation como disciplina própria:

```
Playwright
  ↓
controle determinístico

Browser Use / similares
  ↓
controle agentic

Hermes reasoning
  ↓
planejamento

seams/capabilities
  ↓
execução controlada
```

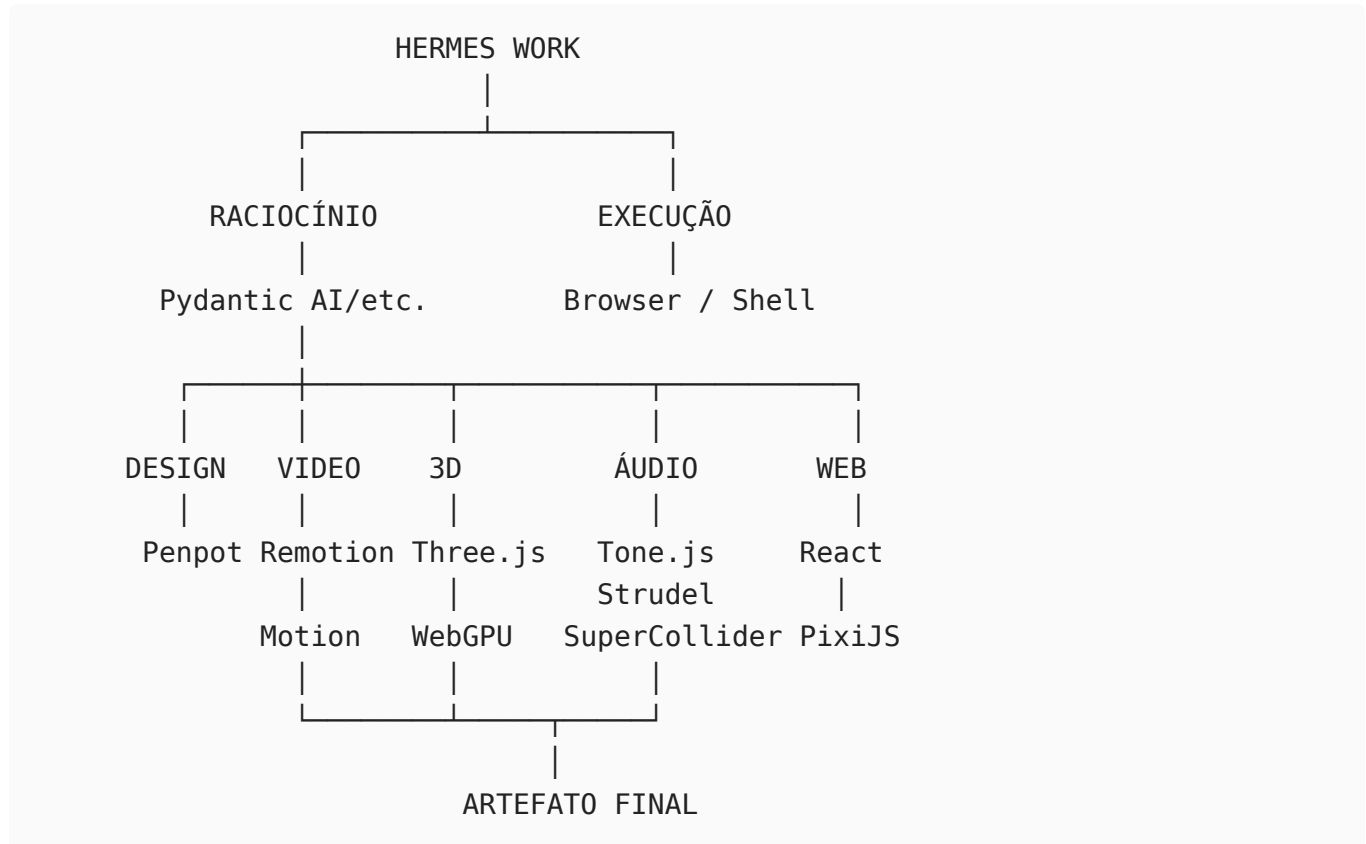
Browser Use está atualmente MIT e pode operar Chrome local, CDP e browsers cloud; sua arquitetura mais recente inclusive migrou para permitir que coding agents executem Python sobre um browser harness em vez de limitar o agente a um menu rígido de ações. [Browser Use](#)

Esse padrão é conceitualmente interessante para o Hermes independentemente de você adotar a ferramenta.

O que eu acho que está surgindo aqui

Seu projeto poderia evoluir para algo consideravelmente maior do que “um agente que usa o computador”.

Pense na seguinte arquitetura:



Nesse modelo, você não instala simplesmente ferramentas.

Você instala **capacidades criativas**.

Hermes poderia receber:

| Crie uma abertura de 8 segundos para o Fórum de IA.

E decidir:

Penpot → composição visual
Three.js → objeto 3D
shader → efeito
Tone.js → sound design
Remotion → timeline
FFmpeg → encoding

Ou:

| Faça um microsite sobre os resultados da SudoExpo.

E usar:

```
dados
→ narrativa
→ React
→ Motion
→ Three.js
→ PixiJS
→ publicação
```

É aqui que sua combinação específica de experiência começa a ficar interessante.

O que eu estudaria primeiro

Não tentaria aprender vinte ferramentas. Eu montaria um **núcleo de alfabetização computacional criativa**:

1. **JavaScript → TypeScript → React**, não para virar frontend developer, mas porque isso se torna a língua franca de Three.js, Remotion, Motion, Tone.js, PixiJS e grande parte da web criativa.
2. **React Three Fiber + Drei + física**, aproveitando imediatamente o conhecimento que Three.js já ativou.
3. **Shaders / GLSL**, porque é provavelmente o maior aumento de poder visual por hora investida.
4. **Remotion + Motion**, conectando diretamente seu passado de motion design ao código.
5. **Tone.js + Strudel**, para incorporar som ao mesmo ambiente criativo.
6. **Penpot + MCP**, pensando não apenas em design, mas em design controlável por agentes.
7. **Pydantic AI/LangGraph**, estudados não como produtos prontos, mas como referências arquiteturais para o Hermes.

E depois eu abriria duas especializações opcionais: **Godot**, se jogos/simulações começarem a exigir uma engine completa; e **SuperCollider/Faust**, se música e sound design deixarem de ser acessórios e se tornarem uma mídia que você queira dominar.

A descoberta maior

Acho que o mais produtivo não é procurar “qual é o próximo Three.js?”.

É perceber o padrão que fez o Three.js ser tão significativo para você.

Você deveria procurar tecnologias que tenham simultaneamente estas propriedades:

```
DOMÍNIO CRIATIVO QUE VOCÊ ENTENDE
      +
REPRESENTAÇÃO EM CÓDIGO
      +
FEEDBACK VISUAL/SONORO IMEDIATO
      +
PARAMETRIZAÇÃO
      +
AUTOMAÇÃO
      +
CONTROLE POR IA
```

Three.js satisfaz isso para 3D.

Remotion satisfaz para vídeo.

Tone.js/Strudel para música.

PixiJS/p5.js para creative coding 2D.

Penpot está caminhando nessa direção para design.

Godot para mundos interativos.

Pydantic AI/LangGraph para agentes.

E aí surge uma tese bastante forte: **o objeto mais importante para você talvez não seja nenhuma dessas ferramentas isoladamente, mas um workstation capaz de dar essas ferramentas a agentes como capacidades interoperáveis.**

Nesse cenário, Hermes Work deixa de ser apenas um navegador autônomo. Ele começa a se tornar uma espécie de **creative operating environment para IA.**

E considero essa direção mais promissora do que simplesmente adicionar Dify ou mais um framework de agentes ao projeto.